PROJECT: MULTI -STAGE CI/CD PIPELINE

Services: AWS EC2, GitHub, Jenkins, Docker, Docker Hub

Project Goal: Design and implement a multi-stage CI/CD pipeline using Jenkins file to automate the build, containerization, and deployment of the Online Bookstore application.

Skills: DevOps automation, containerization, CI/CD pipeline design, and cloud deployment.

Definition: The Multi-Stage CI/CD Pipeline project demonstrates how to automate the software delivery process using Jenkins pipelines defined through a Jenkins file. The pipeline integrates GitHub for source code management, Maven for application build, Docker for containerization, Docker Hub for image storage, and AWS EC2 for deployment. The multi-stage pipeline ensures continuous integration, automated testing and build, container image creation, and continuous deployment of the Online Bookstore application.

Scope of the Project: The scope of this project is to implement a multi-stage CI/CD pipeline using Jenkins file to automate the build and deployment of the Online Bookstore application. The project integrates GitHub for source code management, Maven for building the application, Docker for containerization, Docker Hub for image storage, and AWS EC2 for deployment. This automation helps in achieving faster, reliable, and efficient software delivery.

Methodologies:

1. **Multi-Stage CI/CD Pipeline:** Method used in this project

The method used in this project is a multi-stage CI/CD pipeline implemented using Jenkins file, where the application is automatically built with Maven, containerized using Docker, stored in Docker Hub, and deployed on an AWS EC2 server.

Why this method: The **Multi-Stage CI/CD Pipeline** is best suited because it divides the automation process into **separate stages**, making the pipeline more structured and efficient.

2. **Manual Deployment**: In this method, developers manually build and deploy the application.

Limitations

- Time-consuming
- High chance of human errors
- No automation

Services used in this Project:

1. **AWS EC2 (Elastic Compute Cloud)**

- **Definition:** AWS EC2 is a cloud service that provides virtual servers to run applications and tools on the internet.
- **Purpose:** To provide a cloud server environment to run Jenkins, Docker, and deploy the Online Bookstore application.
- **Role in this Project:** In this project, AWS EC2 hosts the Jenkins server, runs the CI/CD pipeline, and deploys the Docker container for the Online Bookstore application.

2. **GitHub**

- **Definition:** GitHub is a web-based platform used to store, manage, and track source code using version control (Git).
- **Purpose:** The purpose of GitHub in this project is to store the Online Bookstore application code and Jenkins pipeline files.
- **Role in this Project:** In this project, GitHub acts as the source code repository from which Jenkins retrieves the application code to execute the CI/CD pipeline.

3. **Jenkins**

- **Definition:** Jenkins is an open-source automation server used to build, test, and deploy applications through CI/CD pipelines.
- **Purpose:** To automate the multi-stage CI/CD pipeline for building and deploying the Online Bookstore application.
- **Role in this Project:** In this project, Jenkins pulls the code from GitHub, builds the application using Maven, creates the Docker image, pushes it to Docker Hub, and deploys the container on AWS EC2.

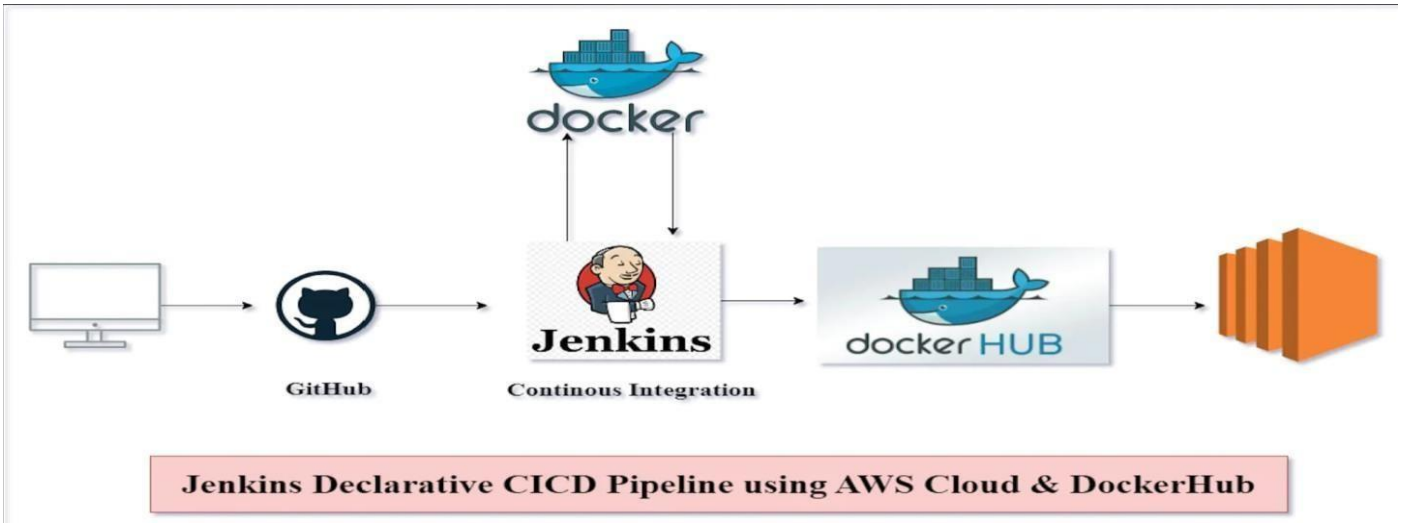
4. **Docker**

- **Definition:** Docker is a containerization platform used to package an application and its dependencies into a portable container.
- **Purpose:** The purpose of Docker in this project is to containerize the Online Bookstore application for consistent and easy deployment.
- **Role in this Project:** In this project, Docker builds the application image and runs the container to deploy the Online Bookstore application on AWS EC2.

5. **Docker Hub**

- **Definition:** Docker Hub is a cloud-based container registry used to store and manage Docker images.
- **Purpose:** The purpose of Docker Hub in this project is to store the Docker image of the Online Bookstore application created by Jenkins.
- **Role in this Project:** In this project, Docker Hub acts as the image repository where Jenkins pushes the Docker image and from where it can be pulled for deployment.

Architecture Diagram



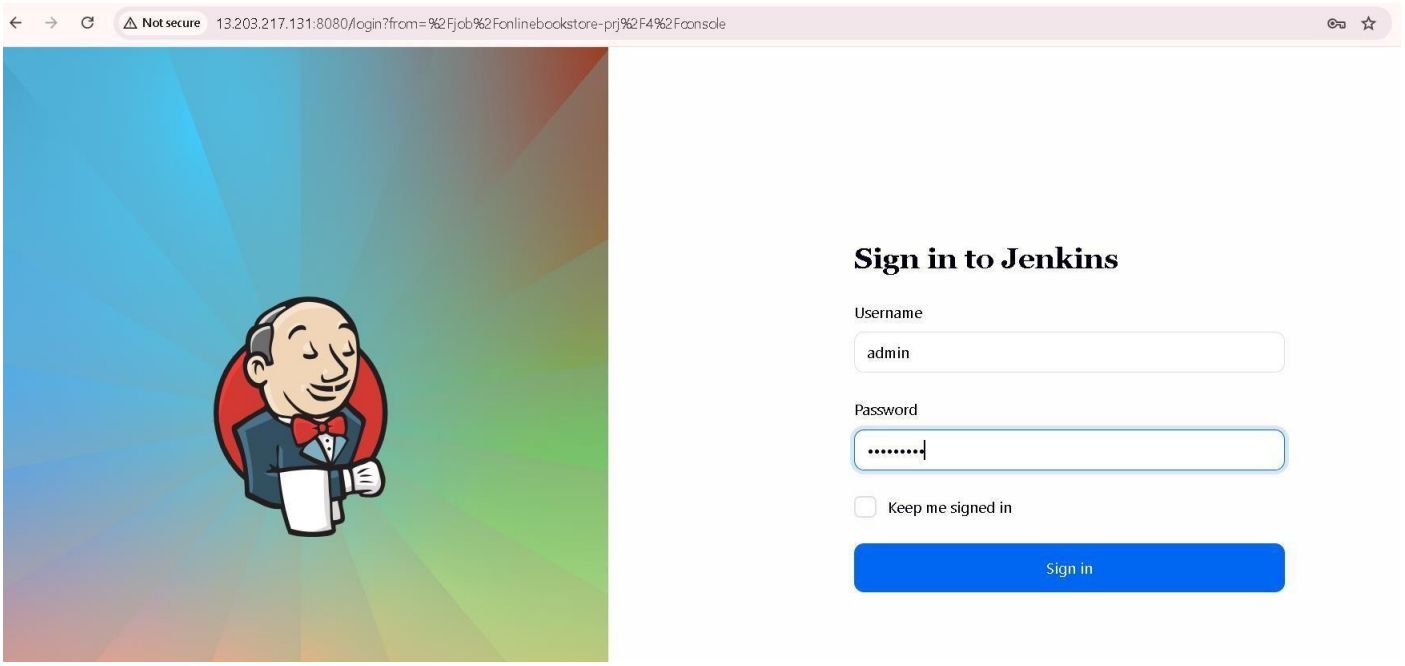
Implementation and Execution of a Multi-Stage CICD Pipeline.

STEP 1: Launch AWS EC2 Instance

- AMI: Amazon Linux
- Instance type: m7iflex.large
- Storage: 30GB
- Security group ports opened:
    - 22 (SSH)
    - 8080 (Jenkins)
    - 80 (Application access)

STEP 2: Start and Configure Jenkins

```
[root@ip-10-0-2-96 ec2-user]# systemctl start jenkins
[root@ip-10-0-2-96 ec2-user]# systemctl enable jenkins
```



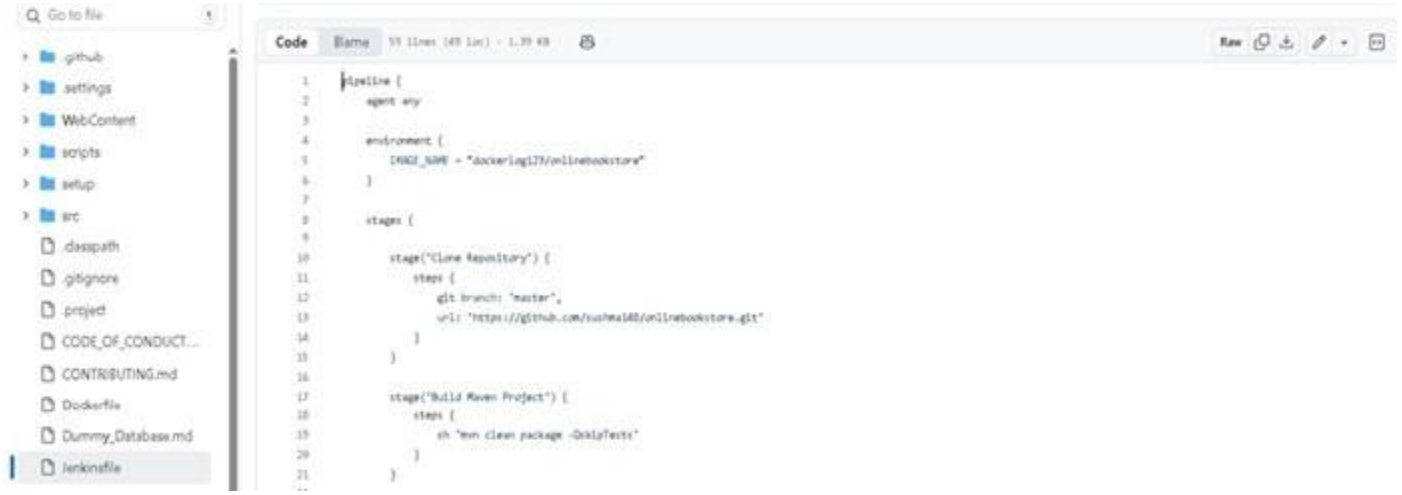
STEP 3: Install Required Jenkins Plugins

- Git Plugin
- Pipeline Plugin
- Docker Pipeline
- Pipeline Stage View

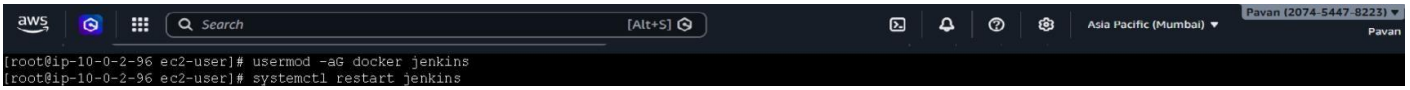
These plugins allow Jenkins to run the CI/CD pipeline.

STEP 4: Implement Jenkins file (Pipeline)

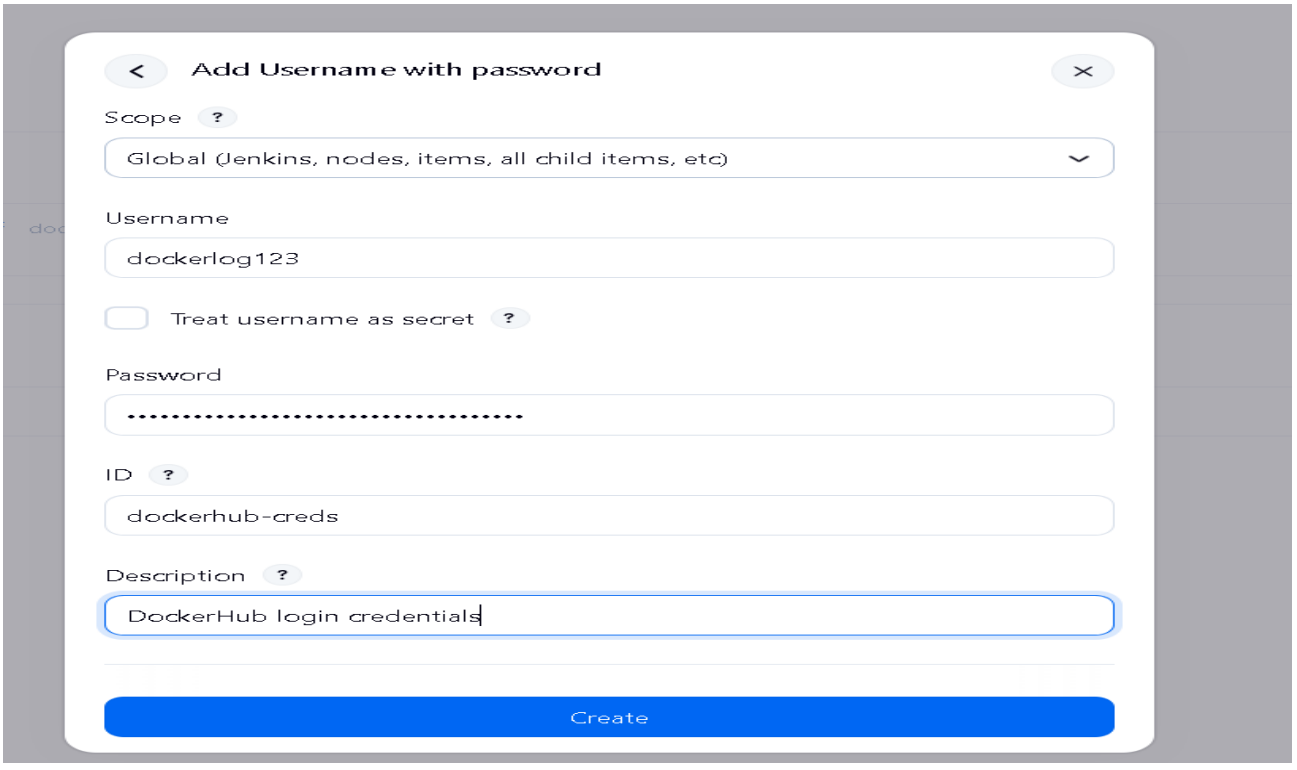
- Created a Jenkins file to define the multi-stage CI/CD pipeline.



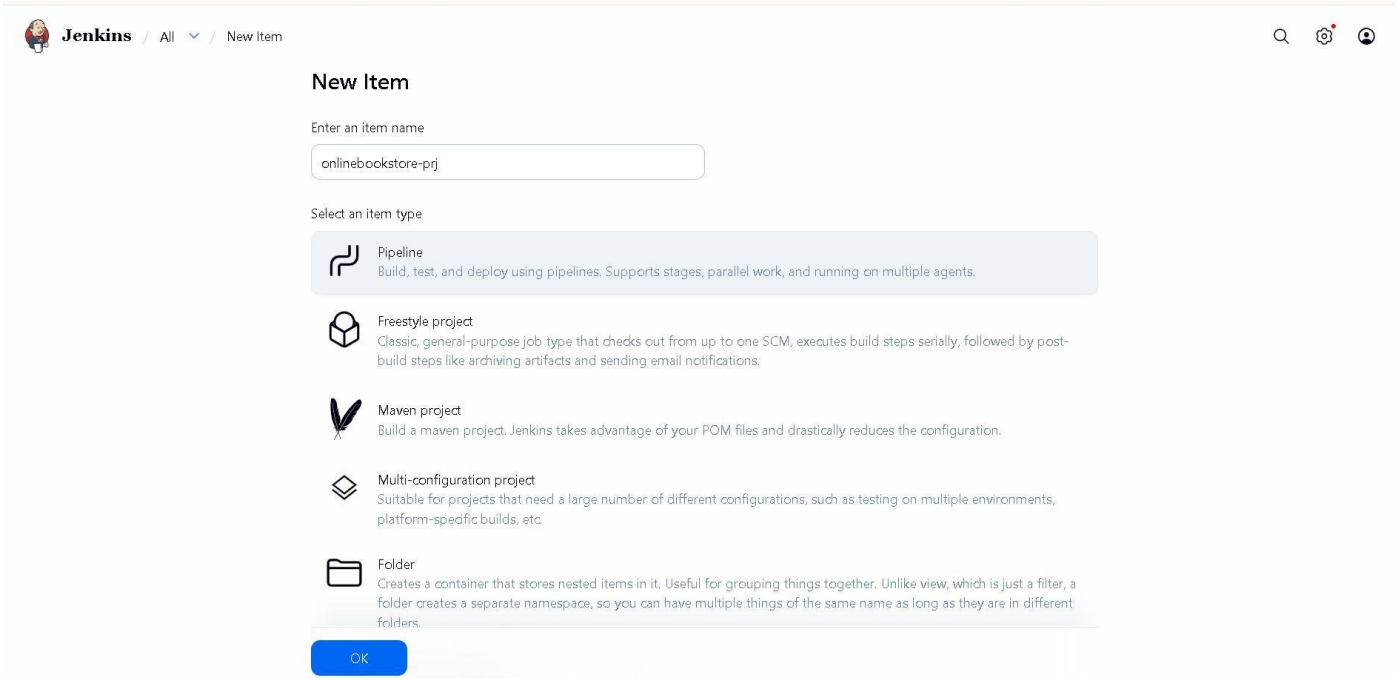
STEP 5: Configure Docker for Jenkins



STEP 6: Add Docker Hub Credentials in Jenkins

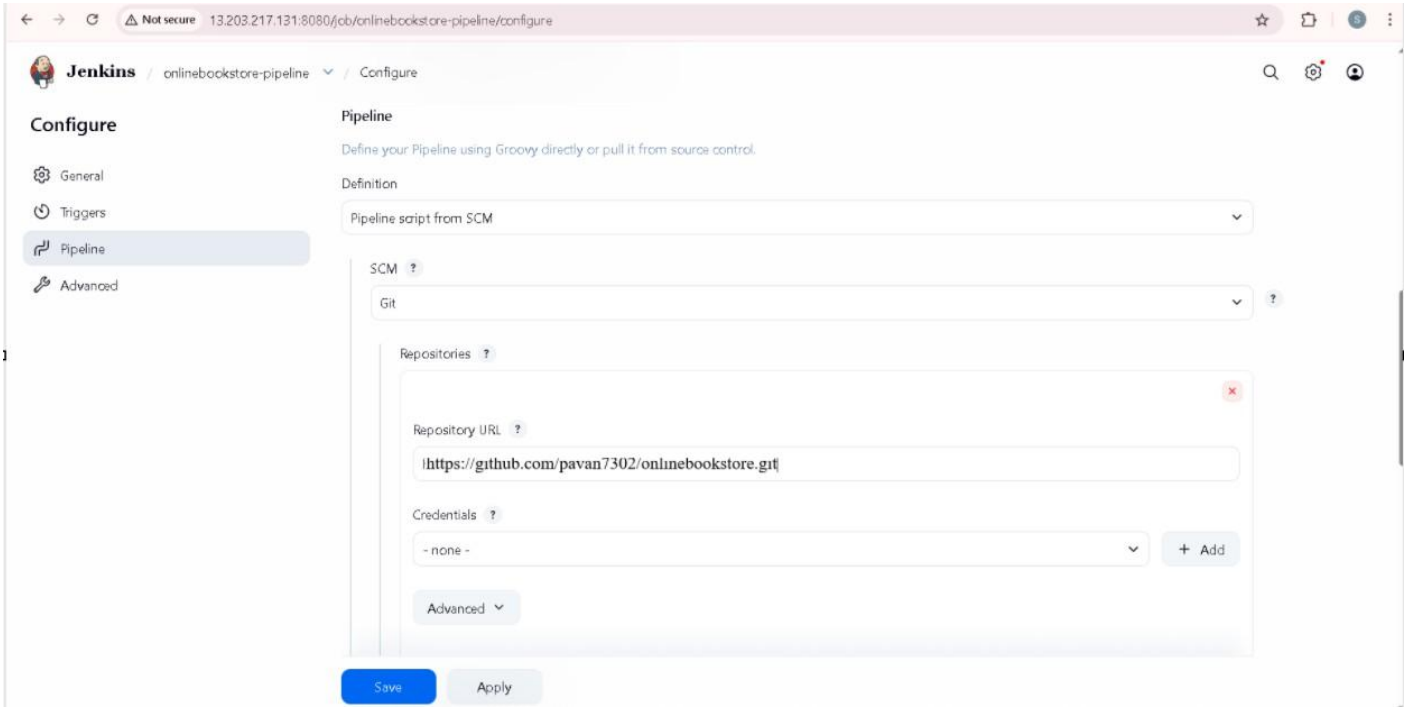


STEP 7: Create Jenkins Pipeline Job



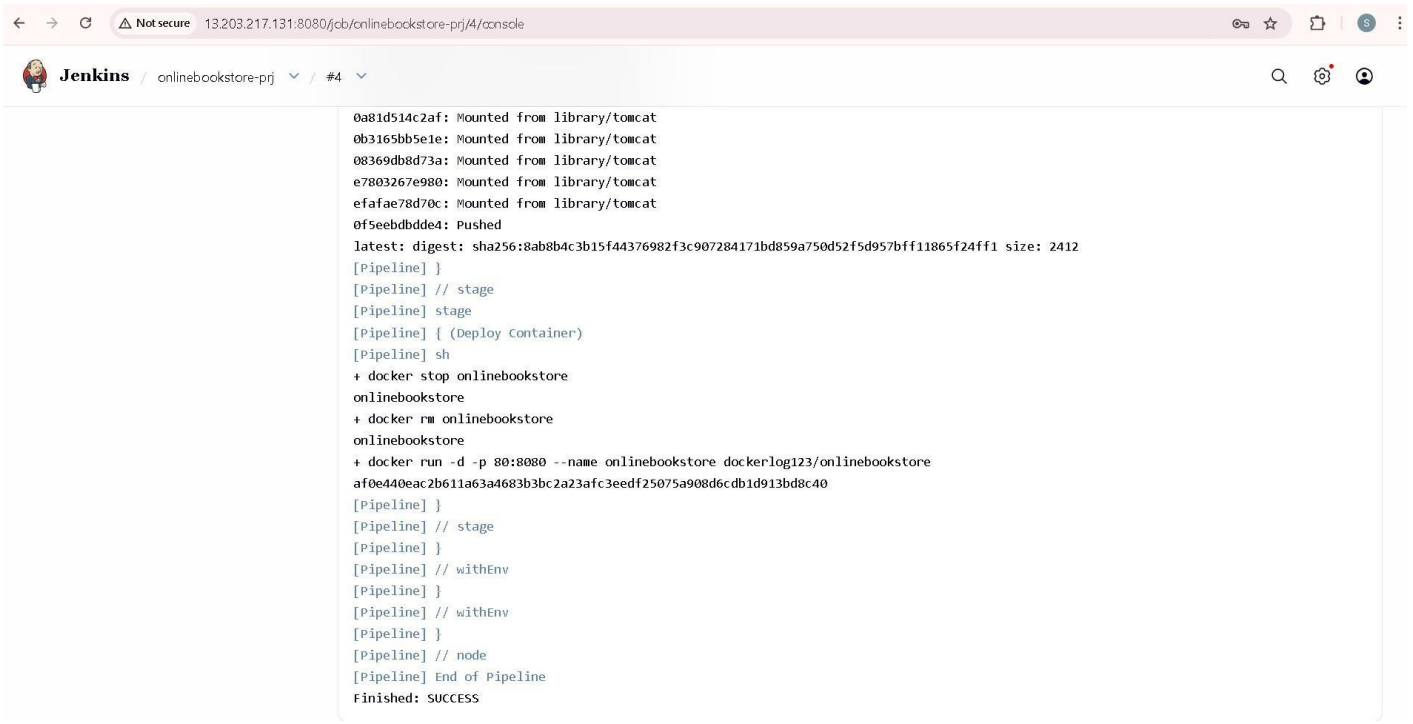
STEP 8: Configured pipeline

- Click Apply and Save

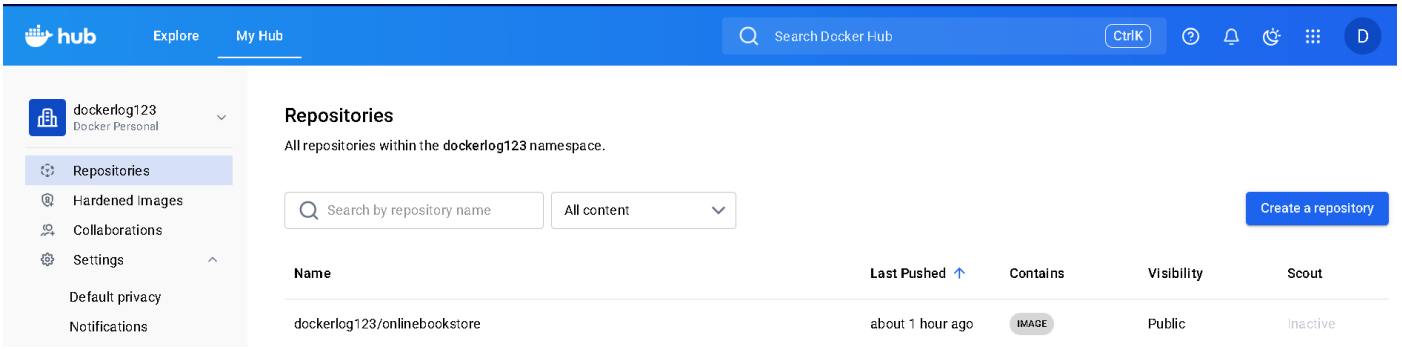


STEP 9: Build the Pipeline Job.

- Build Success.

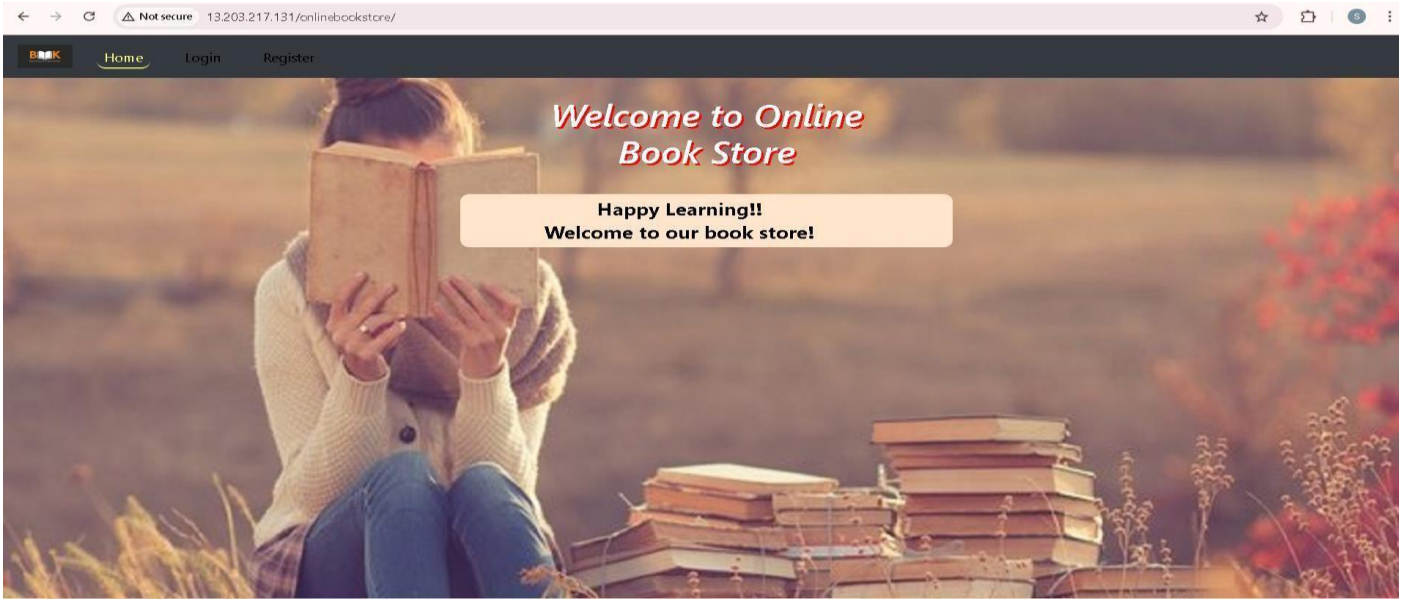


STEP 10: Login to Docker Hub and Push Docker Image.



STEP 11: Access the Application

- Finally, the Online Bookstore application was accessed through the browser.



Troubleshooting:

Issue 1: Jenkins showed a 403 error while saving pipeline configuration due to CSRF protection.
> Solution: Refreshed the Jenkins page and reconfigured the pipeline properly to submit the request.

Issue 2: Docker login failed because Docker Hub no longer accepts normal passwords for CI/CD authentication.

> Solution: Generated a Docker Hub Personal Access Token and used it instead of the password.

Conclusion:

This project successfully implemented a multi-stage CI/CD pipeline using Jenkins to automate the build and deployment of the Online Bookstore application. The pipeline automatically pulls code from GitHub, builds the project using Maven, creates a Docker image, and pushes it to Docker Hub. The application is then deployed as a Docker container on an AWS EC2 instance. This automation reduces manual effort, improves deployment speed, and ensures consistent application delivery. Overall, the project demonstrates how DevOps tools can be integrated to achieve efficient and reliable software deployment.